

허건 HEO GEON

Backend Developer · 백엔드 개발자



백엔드를 만들고, 데이터로 결과를 검증합니다.

Node.js·Spring 백엔드를 중심으로 API 설계, ERD·DB 모델링, 실시간 서버, 데이터 파이프라인을 만듭니다.

엠트리센에서 백엔드 인턴으로 API·DB 구조 설계와 데이터 수집·가공 자동화를 맡았습니다. 팀·개인 프로젝트에서는 실제로 동작하는 서버 구조를 만드는 데 집중했습니다.

구현 결과를 부하 테스트와 데이터 대조로 검증해 믿을 수 있게 만드는 개발을 지향합니다.

학력 · 경력

명지대학교	컴퓨터공학과	2020 ~ 2026
엠트리센	백엔드 개발 인턴	2025 ~ 2026

수상

- ★ 명지대 캡스톤디자인 금상 · 음성인식 키오스크
- ★ 코드잇 데모데이 대상 · 조각집 (ZOGAKZIP)

기술 스택

Backend	Node.js	Express	Spring	FastAPI
Database	MySQL	MongoDB	Redis	
Language	Java	JavaScript	Python	
Tools	Docker	Git	Swagger	

연락처

Email	rjs2337@naver.com
GitHub	github.com/HUHGEON
Blog	huhgeon.github.io
Portfolio	huhgeon.github.io/portfolio

할 수 있는 것

요청이 동시에 몰리는 발급 처리에서 조건부 원자 UPDATE·Redis Lua 선점으로 재고 정확성을 지키고 개선 효과를 부하 측정 수치로 검증할 수 있습니다.	선착순 쿠폰 발급
Spring Batch로 대용량 이력을 재생·대조하는 검증 배치를 만들고 오류를 주입한 데이터로 검증기 자체도 검증할 수 있습니다.	선착순 쿠폰 발급
Spring WebFlux와 Redis Pub/Sub으로 여러 인스턴스에 나뉘어 연결된 사용자끼리 메시지를 중계하는 실시간 서버를 구현할 수 있습니다.	라이브 채팅 서버
LLM 출력을 규칙 라우팅·RAG·스키마 강제로 제어하고 없는 장소 추천·깨진 출력·LLM 장애 시 규칙 폴백으로 대응하는 의도 해석·추천 서버를 설계할 수 있습니다.	날씨 추천 앱·음성인식 키오스크
느린 외부 호출이 커넥션 풀을 고갈시키지 않도록 트랜잭션 경계를 나누고 장애 시 fail-open·폴백 정책을 설계할 수 있습니다.	날씨 추천 앱·라이브 채팅 서버
MongoDB text·TTL·unique 복합 인덱스와 aggregation으로 검색·추천·중복 방지 요구사항을 데이터 모델 수준에서 해결할 수 있습니다.	블로그 플랫폼·음성인식 키오스크

프로젝트

- 01 선착순 쿠폰 발급 시스템** 대규모 트래픽
이력 534만 행을 재생하는 검증 배치 설계·구현, 오류 700건을 주입한 데이터에서 기대 검출 800행을 누락 0·오탐 0으로 확인
- 02 라이브 커머스 채팅 서버** 대규모 트래픽
서로 다른 서버에 연결된 시청자 간 채팅을 Redis Pub/Sub 전달 파이프라인으로 중계
- 03 음성인식 키오스크** LLM 활용
LLM으로 intents를 추출하고 API 서버를 호출하는 오케스트레이터 구조로 NLP 서버 설계
- 04 해야지 (haeyaji)** LLM 활용
로컬 LLM(EXAONE)의 오분류·없는 장소 추천을 규칙 라우팅·RAG·스키마 강제로 줄이고 시나리오 채점 59/80으로 남은 한계까지 확인
- 05 Blog Platform 백엔드** 웹 백엔드
JWT 인증·Multer 업로드를 미들웨어로 분리해 라우터 전반에서 재사용
- 06 엠트리센 인턴·사내 실무** 실무
요청 전처리 → 외부 처리 서버 연동 → 저장·캐싱 백엔드 파이프라인과 Streamlit 데이터 분석·시각화 대시보드 2종 개발

선착순 쿠폰 발급 시스템

조장 · 배치 · 검증 · 스키마 (5인 팀) · 2026.08 · 팀 프로젝트

선착순 요청이 한꺼번에 몰려도 한정 수량 쿠폰을 정확히 발급하고, 그 결과를 데이터로 검증하는 시스템입니다.

Java 21 Spring Boot Spring Batch MySQL Redis Kafka k6

github.com/coupon-yaho 상세 · 시연 영상 ↗

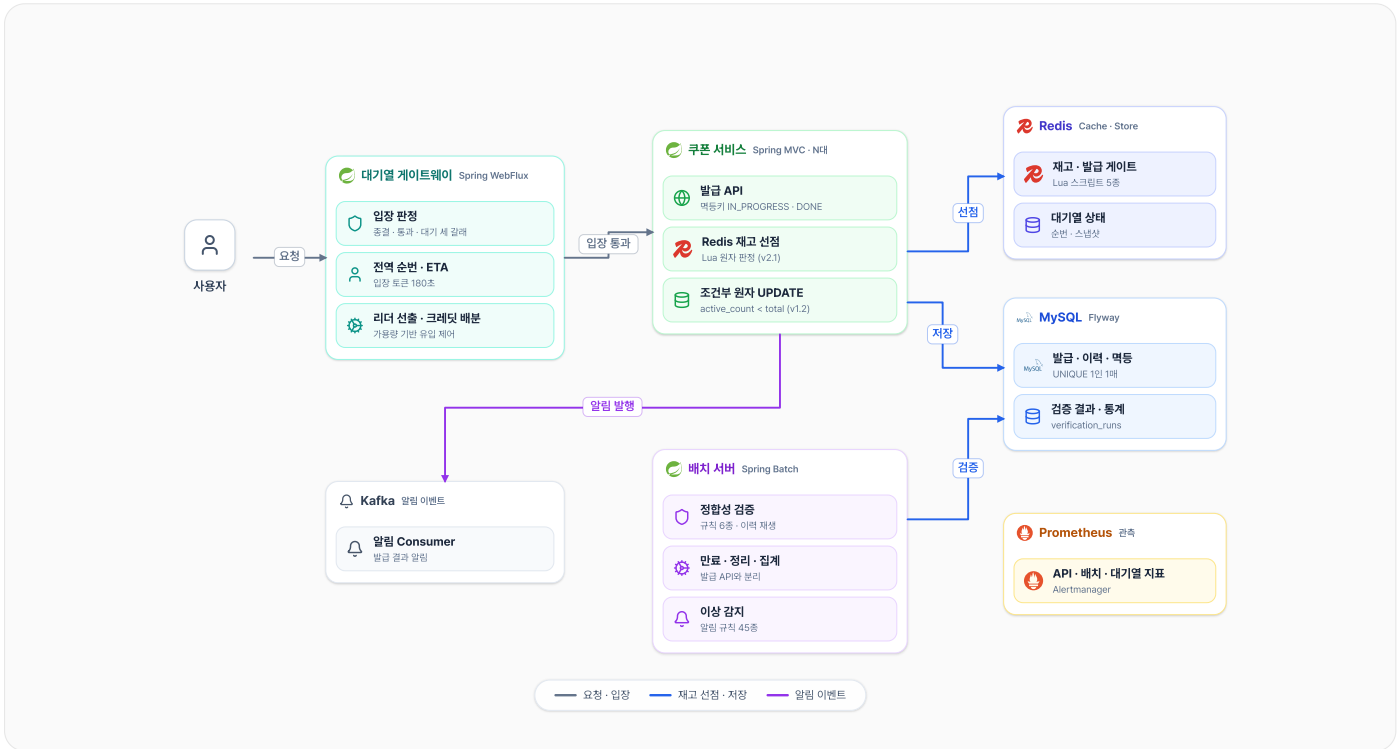
오염 데이터 기대 검출
800 / 800
700건 주입(1개 유형이 규칙 2개에 걸려 기대 800행) · 누락 0 · 오탐 0

500 req/s 중앙값 1초 이상 회차
4/7 → 0/7
7회 중 성공 응답 중앙값이 1초 이상인 횟수

500 req/s 성공 응답 p99
2,240 → 802ms
성공 응답 기준 · 중앙값 1초 이상 회차 제외 · p95 2,160 → 641ms

초과 발급
0건
105회차 전체

LG유플러스 유레카 백엔드 종합 프로젝트에서 5인 팀이 만든 통신사 브랜드이 선착순 쿠폰 발급 시스템(쿠폰 야호~)입니다. 팀은 병목을 측정하면서 MySQL 락, 조건부 원자 UPDATE, Redis Lua 선점, 적응형 대기열 순으로 구조를 바꿔 왔고, 저는 조장으로서 검증 배치, 시드 생성기, 부하 시험 결과의 DB 대조를 맡았습니다.



동작 흐름

- 사용자 요청이 NGINX를 거쳐 대기열 게이트웨이(WebFlux)로 들어오면 즉시 종결 · 무대기 통과 · 대기열 진입 중 하나로 판정
- 통과한 발급 요청은 쿠폰 서비스가 역동키를 IN_PROGRESS로 먼저 기록한 뒤 재고 판정. v1.2는 조건부 원자 UPDATE, v2.1은 Redis Lua 선점
- MySQL이 발급 · 상태 이력 · 사용 실적 · 역동 응답을 저장하고 UNIQUE(coupon_id, member_id)로 1인 1매를 최종 보장
- 발급 결과 알림은 Kafka로 발행
- 배치 서버가 만료 · 정리 · 정합성 검증 · 집계를 발급 API와 분리해 수행, Prometheus가 API · 배치 · 대기열 지표 수집

주요 기능

- **선착순 쿠폰 발급** 역동키(UUID)로 재시도해도 한 장만 발급, 조건부 원자 UPDATE와 UNIQUE(coupon_id, member_id)로 초과·중복 발급 차단
- **적응형 대기열** 평시엔 바로 통과, 혼잡하면 백엔드 가용량에 맞춰 입장량을 조절하고 순번·예상 대기시간·입장 토큰 제공
- **정합성 검증 배치** 검증 규칙 **6종**이 발급 이력을 런타임과 같은 상태 머신으로 재생해 재고 카운터·사용 실적과 대조
- **대용량 시드 생성기** 회원 **100만**·발급 **300만**·이력 **534만** 행을 정상셋과 오류를 섞은 오염셋 두 별로 생성, 시드 고정 RNG로 같은 시드면 같은 데이터 재현
- **배치 이상 감지** 알림 규칙 **45종**(critical **12** · warning **33**), 잡은 성공했지만 데이터가 어긋난 경우도 별도 규칙으로 감지



시연 영상 · 대기열 입장부터 발급 관제 · 검증 배치까지
대기열 입장 → 발급 관제 → 검증 배치
huhgeon.github.io/portfolio/projects/coupon-yaho/

사례 1

오류 700건을 심은 데이터셋으로 검증 배치가 실제로 오류를 찾아내는지 확인, 누락 0 · 오탐 0

문제 원인	- 검증 배치 결과 0건이 오류가 없어서인지 검증기가 못 찾아서인지 구분할 근거가 없음 - 검출 개수만 비교하면 누락과 오탐이 같은 수만큼 섞여도 합계가 기대치와 같아져 통과 - 회원 100만 · 발급 300만 규모를 작은 MySQL 컨테이너에 적재해야 했고 이메일을 난수로 만들면 해시 인덱스 충돌로 적재 전체가 실패할 수 있음
해결 과정	- 시드 생성기로 정상셋(CLEAN)과 오류 7유형 × 100건 = 700건을 심은 오염셋(CORRUPT) 두 벌을 만들고 기대 검출 결과도 기록 - 기대 검출이 800행인 이유: 7유형 중 '사용 취소 이종 기록' 100건은 재고 불일치(V1)와 불법 상태 전이(V4) 두 규칙에 동시에 걸려 200행으로 잡힘 → 600 + 200 = 800행 - 검출 결과를 (finding_type, target_key) 집합 단위로 양방향 대조해 누락과 오탐을 따로 집계 - 시드 고정 RNG와 Feistel 순열로 겹치지 않는 값을 만들어 UNIQUE 충돌 0 보장, 보조 인덱스 없이 적재한 뒤 제약을 한 번에 생성
결과	- CLEAN: 회원 100만 · 발급 300만 · 이력 534만 행에서 검증 규칙 6종 모두 0건, 이력 전체 리플레이 57초 - CORRUPT: 기대 800행 → 검출 800 · 누락 0 · 오탐 0 - CLEAN 생성 · 적재 199.7초, 제약 생성 81초

사례 2

팀에서 재고 잠금을 FOR UPDATE에서 조건부 원자 UPDATE로 바꿔 500 req/s 중앙값 1초 이상 회차 4/7 → 0/7, 성공 응답 p99 2,240ms → 802ms

문제 원인	- 재고 행이 회차당 하나뿐이라 모든 발급이 같은 잠금을 순서대로 통과하고 FOR UPDATE는 조회 시점부터 INSERT까지 임계구간에 넣어 잠금 구간이 김 - 도착률을 올리자 450 req/s부터 성공 응답 중앙값이 1초 이상인 회차가 나왔고(1/7), 500 req/s에선 7회 중 4회로 늘어남 - 지연이 커진 회차도 5xx 없이 느려지지만 해서 가용성 지표만으로는 문제를 잡지 못함
해결 과정	- 검사와 차감을 UPDATE ... WHERE active_count < total_quantity 한 문장으로 합쳐 잠금이 UPDATE에서 시작되게 하고 영향 행이 0이면 소진으로 판정 1인 1매는 UNIQUE(coupon_id, member_id)로 DB에서 최종 보장 - 이후 재고 판정을 Redis Lua 선점으로 옮기고 앞단에 적응형 대기열을 두는 구조로 확장 (아래 버전별 발전 과정)
결과	500 req/s 성공 응답 p99 2,240 → 802ms, p95 2,160 → 641ms(중앙값 1초 이상 회차는 지연 집계에서 제외), 중앙값 1초 이상 회차 4/7 → 0/7 105회차 전체에서 초과 발급 0건

사례 3

측정 환경에 다른 컨테이너가 함께 돌던 것을 발견해 이전 수치를 모두 폐기하고, 외부 요인을 통제한 환경에서 세 잠금 구현을 105회차 재측정

문제 원인	첫 측정 때 다른 프로젝트 컨테이너 6개가 함께 돌고 있어 같은 FOR UPDATE 구현의 300 req/s p99가 6,873ms로 부풀려짐
해결 과정	- 외부 컨테이너를 모두 내리고 회차마다 외부 컨테이너 수와 호스트 CPU 기록 - 세 이미지를 번갈아 실행해 시간대 편향을 없애고 회차마다 예열 후 측정
결과	- 같은 조건 FOR UPDATE 300 req/s p99 6,873 → 222.72ms로 외부 영향 제거, 이전 수치는 전부 폐기 500 req/s 중앙값 1초 이상 회차: FOR UPDATE 4/7 · 조건부 UPDATE 0/7 · Redis 분리 실험 0/7

버전	선택 이유	트레이드오프	수치
v1.1 비관적 락 구현	동시 요청이 회차당 재고 행 하나를 두고 경쟁해 정합성 확보가 먼저 SELECT ... FOR UPDATE로 재고 행을 잠근 뒤 검사·차감. DB가 정확성을 보장하는 가장 단순한 방식이라 이후 버전 비교의 기준선으로 삼음	모든 발급이 같은 잠금을 순서대로 통과하고 INSERT까지 임계구간에 들어가 커넥션 풀을 늘려도 대기가 줄지 않음 → 잠금 구간이 길어 포화 지점이 400~450 req/s에 머물	450 req/s 중앙값 1초 이상 회차 1/7 · 500 req/s 성공 p95 2,160ms · p99 2,240ms · 중앙값 1초 이상 회차 4/7
v1.2 조건부 원자 UPDATE 구현 · 제품 코드	요청마다 잠금 조회와 차감 UPDATE로 두 번 왕복하며 잠금을 오래 잡음 UPDATE ... WHERE active_count < total_quantity 한 문장으로 검사와 차감을 합치고 영향 행이 0이면 소진 판정. 잠금이 UPDATE에서 시작돼 구간이 짧아짐	재고 행 하나를 두고 경쟁하는 구조와 요청별 동기 DB 트랜잭션은 그대로 남음 → 경합 지점이 여전히 DB의 재고 행 하나라 재고 카운트를 DB 밖으로 옮김	500 req/s 성공 p95 641ms · p99 802ms · 중앙값 1초 이상 회차 0/7
v2.1 Redis Lua 원자 선점 구현	DB에 직렬 구간이 남아 있으면 커넥션을 늘려도 처리량이 늘지 않음 Lua 스크립트 5종(claim · complete · compensate · restore · reclaim)이 회차 상태 · 등급 · 1인 1매 · 재고를 한 번에 판정하고 차감. 발급은 선점 → 트랜잭션 → 완료 CAS 세 단계로 분리	저장소가 둘로 나뉘어 선점 직후 중단되면 Redis가 DB보다 앞서감. 요청 토큰으로 보상, Redis 유실 시 DB 기준으로 재구성, Redis↔DB 격차 모니터링 지표 추가 → DB 경합은 줄었지만 순간 유입 자체와 건별 영속화는 그대로 남음	재고 UPDATE를 별도 트랜잭션으로 분리한 실험(v2-split): 500 req/s 성공 p95 562ms · p99 748ms · 중앙값 1초 이상 회차 0/7 · 발급당 DB 쓰기 7행 → 3행 (실험 · 미커밋)
v2.2 적응형 대기열 핵심 경로 구현	순간적으로 몰리는 유입이 애플리케이션과 DB에 함께 과부하를 줌 게이트웨이를 별도 서비스로 분리한 이유는 세 가지. 경계 (입장만 담당, 재고 차감은 쿠폰 서비스만) · 스택(입장은 WebFlux, 발급은 MVC + JPA라 한 프로세스에 두면 블로킹이 이벤트 루프를 막음) · 장애 격리(뒷단이 멈춰도 대기열 유지)	입장은 순수만 보장할 뿐 발급은 보장하지 않음. 컴포넌트가 늘고 성공 요청마다 동기 DB 커밋도 여전히 필요	페이지 게이트 7단계 통과 · 요청 경로 Redis 명령 0건 · 오버헤드 p99 < 5ms · 크레딧 초과 배분 0/10만 회 · 매진 후 동시 조회 1만 → 뒷단 1건

잠금 방식별 부하 측정 ms

같은 커밋에서 재고 잠금 방식만 바꿔 재고 10,000 · 요청 20,000 조건으로 도착률마다 7회씩 측정했습니다. p95 · p99(ms)는 성공 응답 중앙값이 1초 미만인 회차들의 중앙값이며, 1초 이상인 회차 수는 오른쪽 열에 따로 표시했습니다.

도착률	구현	p95	p99	1초 이상 회차
300/s	v1.1 FOR UPDATE	50.10	222.72	0/7
	v1.2 조건부	28.12	158.64	0/7
400/s	v1.1 FOR UPDATE	409.35	481.83	0/7
	v1.2 조건부	190.41	264.03	0/7
450/s	v1.1 FOR UPDATE	562.01	648.15	1/7
	v1.2 조건부	368.66	575.49	0/7
500/s	v1.1 FOR UPDATE	2,159.79	2,239.89	4/7
	v1.2 조건부	640.92	802.16	0/7

k6 constant-arrival-rate · 로컬 Docker(API 2대 · MySQL 단일) · 외부 컨테이너 0개 · 회차마다 상태 롤백. 1초 이상 회차는 성공 응답 중앙값이 1,000ms 이상인 회차로, 지연 집계에서 제외했습니다. 따라서 FOR UPDATE 450 · 500 req/s의 p95 · p99는 나머지 회차만의 값이며 실제 꼬리 지연은 더 나쁩니다.

배운 점

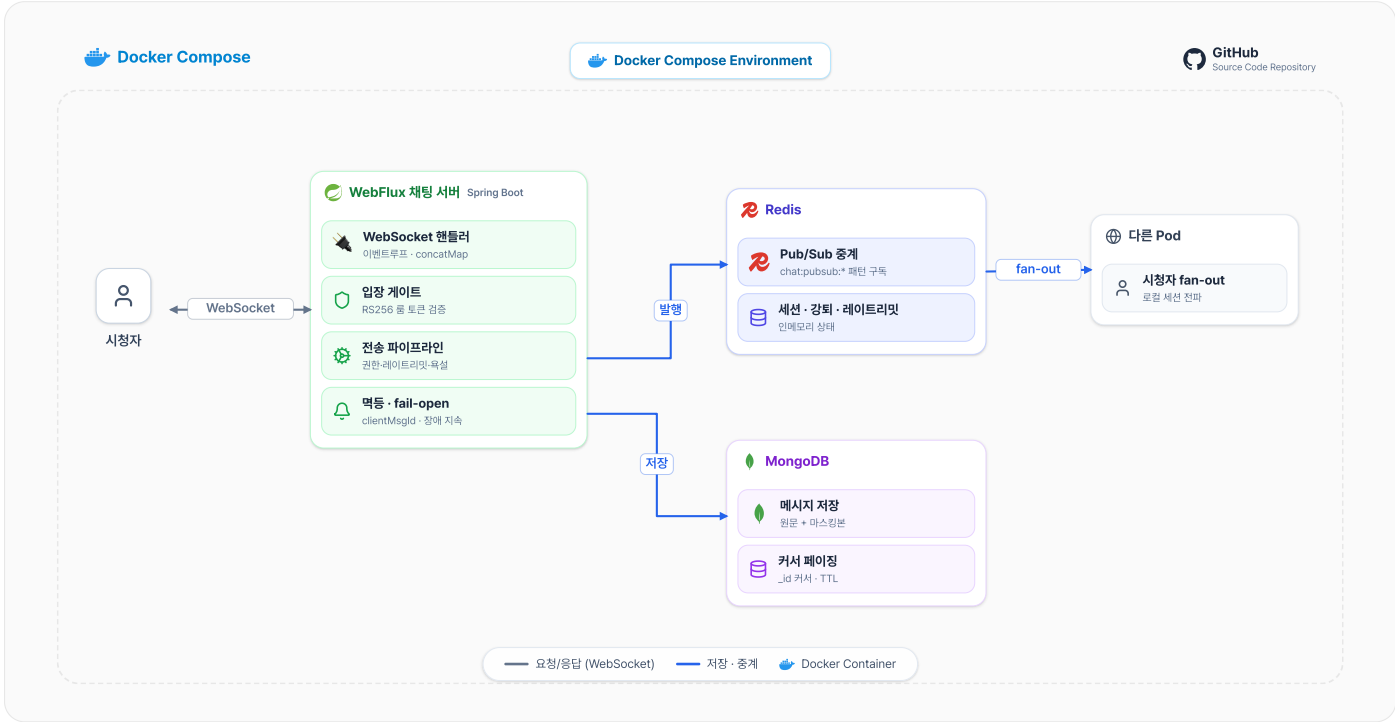
평상 부하(300 req/s)에서는 세 구현의 성공 응답 중앙값이 3.4~4.0ms로 차이가 드러나지 않았습니다. 개선 효과는 평소 지연을 줄인 것이 아니라 포화 지점을 끌어올린 데 있었고, 이 차이는 지표를 제대로 고르고 측정 환경을 통제해야만 보인다는 것을 배웠습니다. 검증도 마찬가지로 '0건'이라는 결과는 오류를 심어 두고 전부 찾아냈을 때만 믿을 수 있다는 기준을 세웠습니다.

라이브 커머스 채팅 서버

sapari(라이브 스트리밍·커머스) 채팅 서버 담당 · 진행 중 · 도메인 확장 예정 · 2026.06 ~ 진행 중 · 팀 프로젝트

여러 Pod에 흩어진 시청자에게 메시지를 실시간으로 전달하는 라이브 채팅 서버입니다.

Java 21 Spring WebFlux WebSocket Redis MongoDB github.com/sago-panda/sapari-be/tree/feature/SPR-16 상세 페이지 ↗



문제

- WebSocket은 stateful이라 시청자가 여러 Pod에 나뉘고, A Pod에서 보낸 메시지를 B Pod 시청자에게도 실시간으로 전달해야 함
- 단일 이벤트 루프를 블로킹하지 않으면서 재전송·중복 프레임·파이프라이닝을 이용한 레이트리밋 회피를 막아야 함
- 채팅 본문·clientMsgId·프레임 같은 신뢰할 수 없는 입력을 이용한 욕설 우회·로그 위조를 막아야 함

결과

- 어느 Pod에 접속해도 같은 방 메시지를 받는 stateless 수평 확장 구조, 한 JVM에서 Pod 2개를 모사한 TestContainers(Redis) 통합 테스트로 Pod 간 전달 확인 (Pub/Sub 특성상 전달은 best-effort이고 재접속 시 누락 복구는 미구현)
- Redis 장애(fail-open)·poison-message(개별 skip)·재전송(먹통 처리) 세 상황 모두에서 전송 파이프라인이 멈추지 않도록 구성

해결

- 비용 0인 검사는 앞에, Redis I/O는 뒤에 두는 순서(방 생존 → 검증 → 권한 → 강퇴 → 레이트리밋 → 욕설 마스크 → 저장 → 발행)로 전송 파이프라인 구성
- Redis Pub/Sub 패턴(chat:pubsub:*)을 hot 스트림으로 상시 구독해, 방별 lazy 구독에서 생기는 '구독 완료 전 메시지 유실' 레이스 없이 Pod 간 fan-out
- Aho-Corasick 1-패스 욕설 탐지로 특수문자 삽입·인접 토큰 결합 우회까지 잡고, '시발점' 같은 일반 단어는 화이트리스트로 보존
- clientMsgId + Mongo partial unique index로 먹통 처리(persist-then-publish)해 재전송 시 중복 발행을 막고 기존 메시지로 ack
- Redis 장애로 강퇴·레이트리밋 조치가 실패해도 채팅 허용(fail-open), onErrorResume 람다 안 throw 금지와 시간 기반 로그 스로틀로 fail-open 정책이 뒤집히지 않게 함
- chat-api(계약) ↔ chat-core(도메인·인프라)를 핵사고날 구조로 분리하고 ArchUnit으로 강제, 인바운드는 concatMap으로 순차 처리

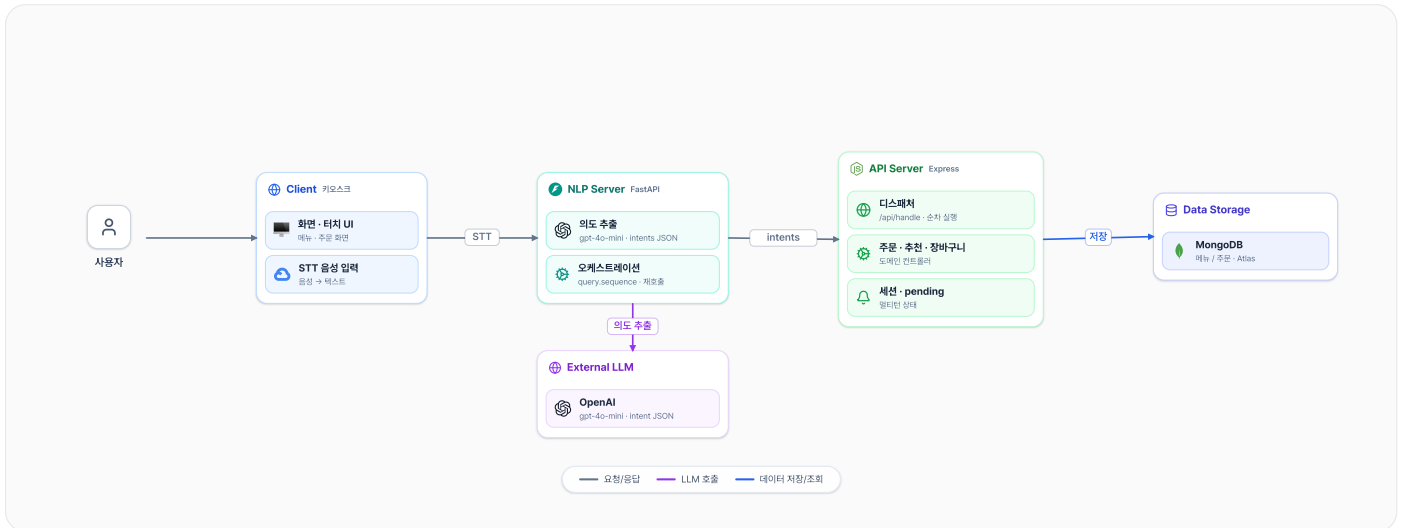
음성인식 키오스크

★ 명지대 캡스톤디자인 금상

NLP 서버 설계·주도 · API 연동 · 2025.03 ~ 2025.06 · 팀 프로젝트

자유발화 한 문장에 담긴 여러 요청을 순서대로 처리하는 음성 주문 백엔드입니다.

FastAPI gpt-4o-mini Express MongoDB Swagger

github.com/Say-It-It-s-OK 상세 · 시연 영상 ↗

문제

- 한 문장에 섞인 여러 요청('카페라떼 빼고 아메리카노 추가하고 결제')를 순서대로 실행해야 함
- 주문 시 필수 옵션(온도·크기 등)이 빠져도 바로 실패로 끝내지 않고 다음 발화로 채워야 함

결과

- 복합 요청 처리: 한 문장에 담긴 여러 의도를 순서대로 실행
- 멀티턴 처리: 빠진 옵션을 후속 발화로 채우며 주문 진행
- 추천부터 장바구니 수정·결제까지 이어지는 키오스크 백엔드를 팀 프로젝트로 완성
- 결제 시 메뉴별 수량을 그룹핑해 Orders 컬렉션(MongoDB Atlas)에 실제 저장, API는 Swagger로 문서화 (팀 API 서버)

해결

- LLM 시스템 프롬프트에 intents 9종·filters·옵션 정규화(샷 표현·온도 수식어 분리 등) 규칙을 정의하고 gpt-4o-mini로 JSON 추출
- intents가 있으면 query.sequence로 묶고 API 서버가 for-loop로 순차 실행, 중첩 응답은 results만 펼쳐(flatten) 병합
- 옵션이 빠지면 pending으로 세션에 보관, 옵션만 담긴 후속 발화가 오면 남은 옵션을 계산해 다 채워졌을 때 장바구니에 담기 (팀 API 서버와 연동)
- 추천은 MongoDB aggregation으로 주문↔메뉴 \$lookup·인기순 집계, 이미 추천한 항목은 \$nin으로 제외해 중복 추천 방지 (팀 API 서버)
- menus·query·recommend·order·cart API를 Swagger(swagger-jsdoc)로 문서화해 프론트·NLP와 연동 규격 공유 (팀 API 서버)



시연 영상 · 음성 대화로 추천부터 결제까지

인기 메뉴 보여 줘 → 아이스 아메리카노 미디움 사이즈로 주문해 줘 → 미디움으로 해 줘 → 결제해 줘

huhgeon.github.io/portfolio/projects/voice-kiosk/

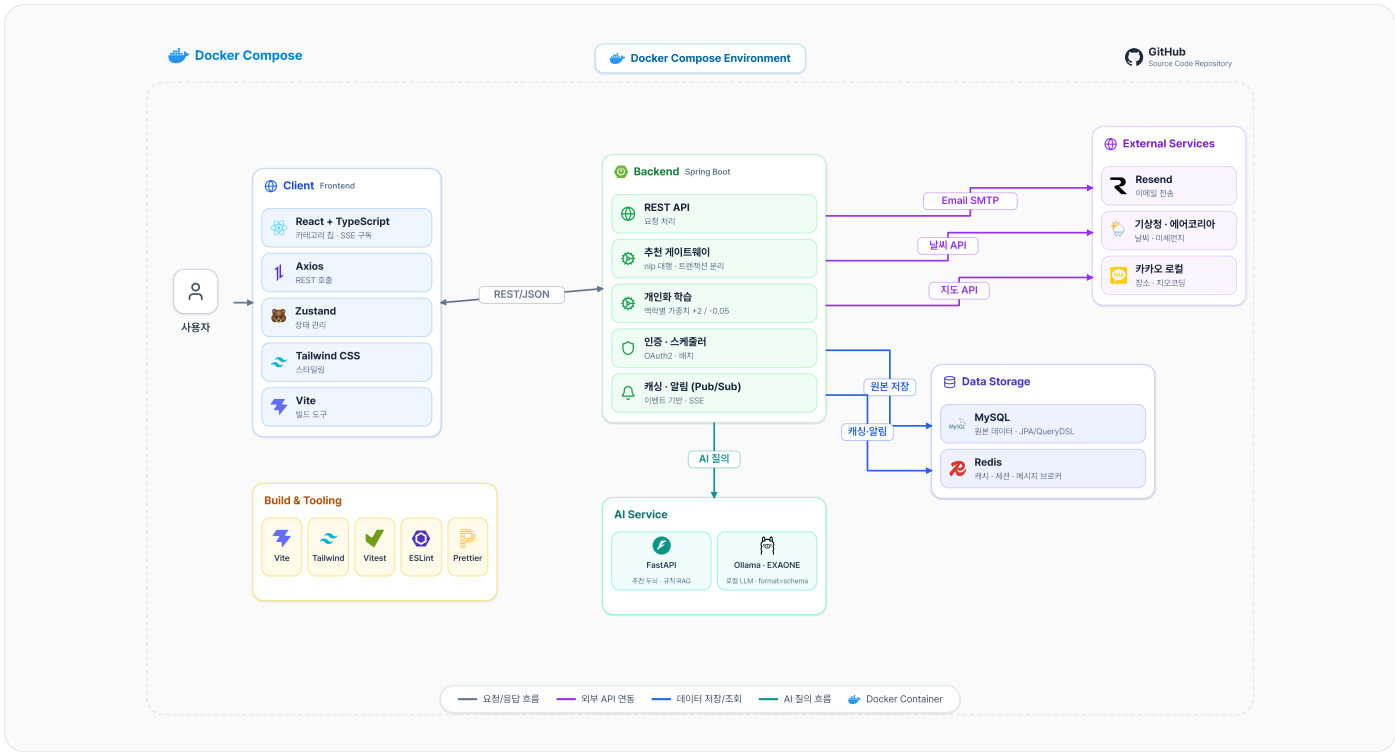
해야지 (haeyaji)

NLP 추천 서버 단독 개발 · 백엔드 날씨/추천/개인화/알림 도메인 · 2026.06 ~ 2026.07 · 팀 프로젝트

작은 로컬 LLM에 규칙과 RAG를 붙여 '오늘 뭐 할지' 추천이 덜 들쭉날쭉하게 만든 추천 서버입니다.

FastAPI Ollama · EXAONE Spring Boot MySQL Redis QueryDSL

github.com/haeyaji 상세 · 시연 영상 ↗



문제

- 작은 로컬 LLM(2.4~7.8b)의 막연한 발화 오분류-없는 가제를 지어내는 환각-구조화 JSON 형식 깨짐
- LLM 응답이 최대 20~45초 걸리는 동안 트랜잭션-커넥션을 붙잡으면 커넥션풀 고갈, 외부 API 호출 폭주와 알림 유실도 막아야 함

결과

- nlp 시나리오 자동 채점 59/80 통과 (2026-07-06 기준: 날씨-위치 19/20, 막연한 요청 17/20, 상세 추천 17/20, 도메인 밖 거절 6/20)
- 도메인 밖 거절 실패 14건(요리법·번역·운세 등에서 거절이 동작하지 않음)을 확인해 LLM 호출 전 규칙 라우팅을 보강했고, 7.8b 모델로는 작정하고 시도하는 사회공학을 완전히 막을 수 없다는 한계를 PR에 명시 (보강 후 동일 평가셋 재측정 결과는 기록 없음)
- LLM-네트워크 없이 도는 결정적 테스트(nlp 119개·be 36개)로 규칙-라우팅·폴백·날씨 변환·알림 역등성 회귀 검증

해결

- LLM은 활동·검색어만 정하고, 카카오 후보 목록을 넣어 그 안에서만 고르게 하는 RAG로 없는 장소 추천 방지(후보에 없으면 장소 연결 해제)
- 막연한 표현·재요청·부정·인사·도메인 밖 거절·프롬프트 인젝션은 LLM 호출 전에 규칙으로 확정, 카테고리는 날씨×시간대 가점으로 점수화
- Ollama 호출에 format=<Pydantic schema>로 출력 스키마를 강제하고, 실패·타임아웃·깨진 JSON에도 500 대신 규칙 폴백 반환
- 추천 게이트웨이 트랜잭션을 분리해 느린 nlp 호출 전에 DB 조립과 커넥션 반납을 끝내고, LLM 응답 중 커넥션풀 고갈 방지
- 고른 것 +2 / 함께 노출됐지만 안 고른 것 -0.05를 (날씨×시간대) 맥락별로 원자적으로 누적하고 주간 decay를 적용하는 개인화 가중치 학습
- 위경도→격자·중기 지역코드·최근접 측정소 변환, 발표시각 자동 계산, Redis 캐시, 외부 API 장애 때도 응답을 유지하는 기상청·에어코리아 날씨 중계 파이프라인 구축



시연 영상 · 날씨 기반 장소 추천부터 일정 추가까지
날씨 홈 → 지도 추천 → 추천 도우미 → 일정 추가
huhgeon.github.io/portfolio/projects/haeyaji/

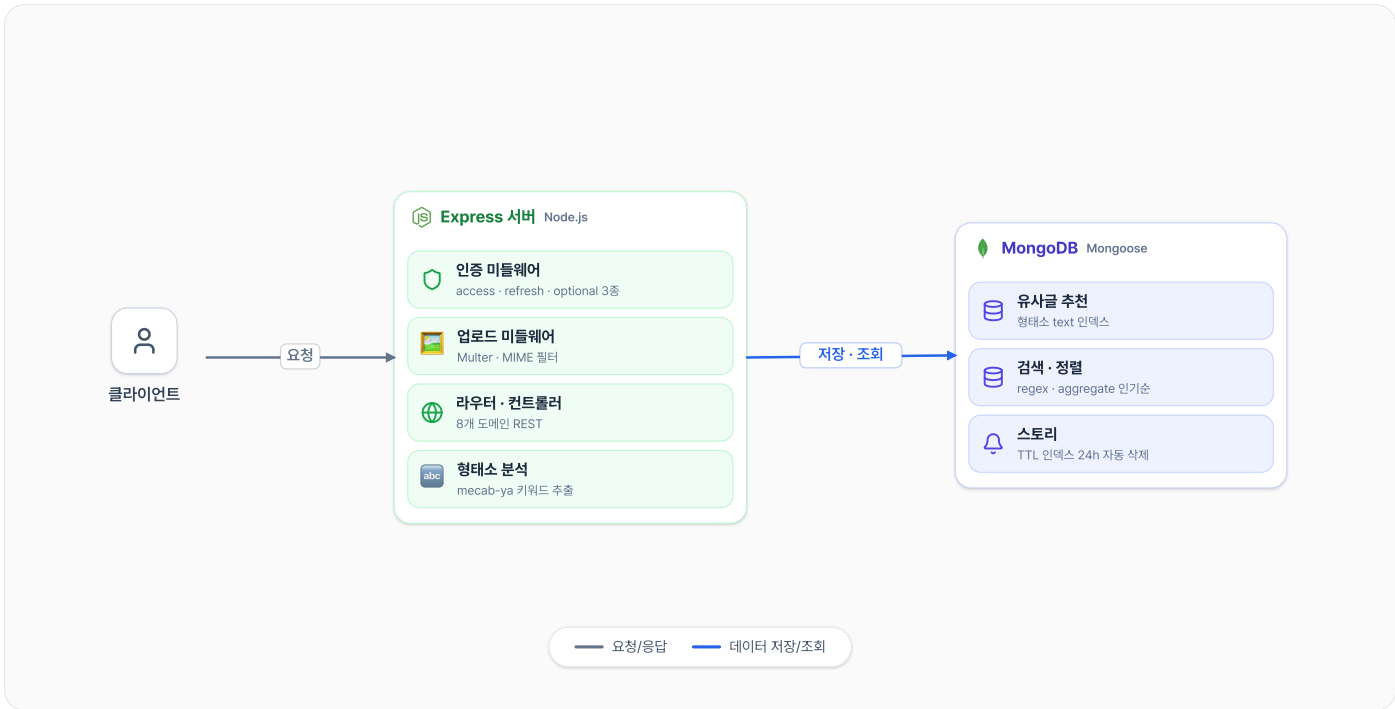
Blog Platform 백엔드

Express · MongoDB 백엔드 (개인) · 2025.08 · 개인 프로젝트

엔드포인트마다 다른 인증 요구는 미들웨어로, 한국어 검색은 형태소 분석으로 해결한 블로그 백엔드입니다.

Express MongoDB JWT mecab-ya Multer

github.com/HUHGEON/Blog-Platform 상세 · 시연 영상 ↗



문제

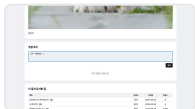
- 조사-어미가 붙는 한국어는 단순 문자열 매칭으로 '비슷한 글 추천'에 쓸 만한 유사도가 나오지 않음
- 엔드포인트마다 인증 요구 수준이 달라(작성=필수, 상세 조회=비로그인 허용+로그인 시 좋아요 상태, 갱신=refresh만) 인증 로직 하나로 처리 불가

결과

- 인증(3종)·게시글·댓글·좋아요·팔로우·쪽지·스토리 8개 도메인 REST API 백엔드 완성
- 형태소 기반 유사글 추천·조회수+좋아요+댓글 합산 인기순 정렬
·24시간 TTL 스토리 등 기본 CRUD를 넘어서는 기능을 인덱스
·aggregate로 구현

해결

- 인증을 authenticateToken / optionalAuth / authenticateRefreshToken 3종으로 분리, 라우트별 요구 수준에 맞춰 적용
- 토큰 payload의 type(access/refresh)으로 종류 구분, 발급·검증 로직은 JWT 유틸로 모듈화
- mecab-ya로 게시글의 한국어 명사를 추출하고 제목 키워드에 3배 가중치를 준 뒤 analyzed_keywords_text의 text 인덱스로 \$text + textScore 유사글 추천
- 컬렉션당 text 인덱스는 1개뿐인데 추천용이 이미 사용 → 일반 제목·본문 검색은 regex \$or로 분리해 인덱스 충돌 회피
- 비밀번호는 스키마 pre-save 훅에서 bcrypt(cost 12)로 해싱, 좋아요는 unique 복합 인덱스로 중복 방지·\$inc로 카운터 동기화
- 스토리는 TTL 인덱스로 24시간 뒤 자동 삭제



시연 영상 · 스토리 · 검색 · 유사 글 · 팔로우 · 쪽지
스토리 → 검색 · 상세 → 팔로우 · 쪽지 → 새 글 작성
huhgeon.github.io/portfolio/projects/blog-platform/

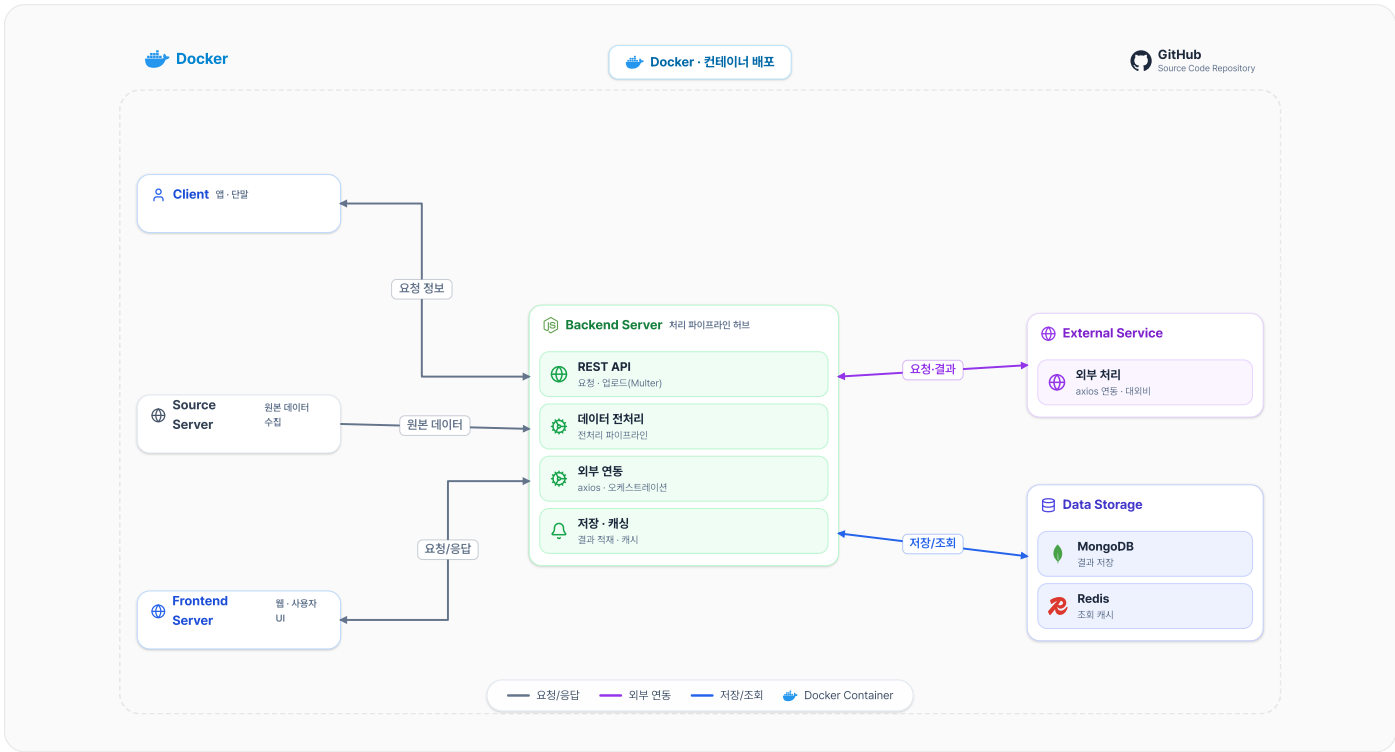
엠트리센 인턴 · 사내 실무

백엔드 · 데이터 분석 자동화 · QA (사내 프로토타입) · 2025.08 ~ 2026.01 · 엠트리센 인턴

백엔드 프로토타입 개발부터 데이터 분석 자동화, QA까지 맡은 사내 실무입니다.

Node.js Express Python Streamlit pandas MongoDB Redis Jest

사내 · 비공개 저장소 상세 페이지 ↗



사내 실무 · 기술 중심

백엔드 프로토타입 · 처리 파이프라인

업로드 데이터 전처리 → 외부 처리 서버 연동 → 결과 저장·캐싱으로 이어지는 백엔드 파이프라인 구현, 업로드·전처리·외부 연동·저장을 서비스 레이어로 분리

Node.js Express MongoDB Redis Docker

데이터 분석·시각화 대시보드 ①

기획팀에서 부탁받은 조사용 엑셀 작업이 반복된다는 점을 보고 스스로 자동화. 엑셀 데이터 로드·가공·통계 분석·차트 시각화를 하는 Streamlit 대시보드로 수작업 분석 대체

Python Streamlit pandas numpy matplotlib scipy

데이터 분석·시각화 대시보드 ②

같은 기획팀 조사 업무에서 기간·조건별 엑셀 집계·분석을 Altair 차트 기반 Streamlit 대시보드로 자동화. 데이터 로드·가공·시각화는 유틸 모듈로 분리

Python Streamlit pandas numpy Altair

테스트 · QA

- 자체 프로젝트(백엔드·데이터 도구): 테스트 케이스를 직접 설계·수행해 동작 검증. 백엔드는 Jest 단위·통합 테스트, 데이터 도구는 입력·경계(엣지) 케이스로 결과 검증
- 타사 앱 QA: 사내에서 다른 회사 앱을 직접 테스트하고 결과를 테스트 보고서로 정리·보고하는 업무를 자주 담당

※ 도메인·세부 기능·정량 성과는 대외비로, 사용 기술과 구조만 기재.